ENV 797 - Time Series Analysis for Energy and Environment Applications | Spring 2026

Assignment 7 - Due date 03/06/26

Sylvia Hipp

Directions

You should open the .rmd file corresponding to this assignment on RStudio. The file is available on our class repository on Github. And to do so you will need to fork our repository and link it to your RStudio.

Once you have the file open on your local machine the first thing you will do is rename the file such that it includes your first and last name (e.g., "LuanaLima_TSA_A07_Sp26.Rmd"). Then change "Student Name" on line 4 with your name.

Then you will start working through the assignment by **creating code and output** that answer each question. Be sure to use this assignment document. Your report should contain the answer to each question and any plots/tables you obtained (when applicable).

When you have completed the assignment, **Knit** the text and code into a single PDF file. Submit this pdf using Sakai.

Packages needed for this assignment: "forecast","tseries". Do not forget to load them before running your script, since they are NOT default packages.\

Set up

```
#Load/install required package here
library(tidyverse)
library(forecast)
library(tseries)
library(here)
library(cowplot)
library(Kendall)
library(glue)
```

Importing and processing the data set

Consider the data from the file "Net_generation_United_States_all_sectors_monthly.csv". The data corresponds to the monthly net generation from January 2001 to December 2020 by source and is provided by the US Energy Information and Administration. **You will work with the natural gas column only**.

```
# import raw data
net_gen_raw <- read.csv(here("Data/Net_generation_United_States_all_sectors_monthly.csv"),
                        skip = 4) %>%
```

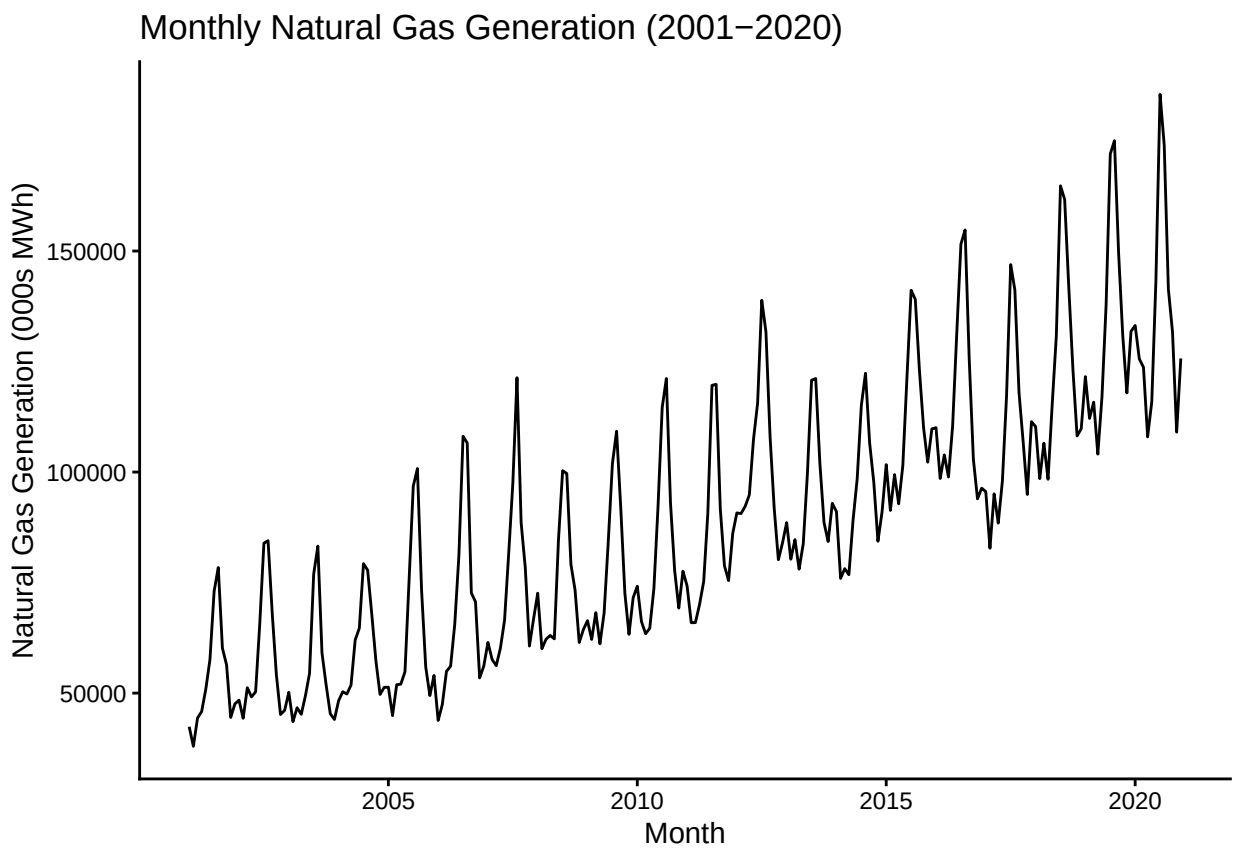
```r
  janitor::clean_names()               # clean up names

# trim dataset to just natural gas generation
net_gen_ng <- net_gen_raw %>%
  tibble() %>%
  select(month, natural_gas_thousand_megawatthours) %>%
  rename("ng_thousand_mwh" = natural_gas_thousand_megawatthours) %>%
  mutate(month = my(month)) %>%    # convert month to date
  arrange(month)                   # make sure data is in the correct order

# make sure there are no missing observations
first <- first(net_gen_ng$month)
last <- last(net_gen_ng$month)

# join sequence of all months in date range
net_gen_ng <- full_join(net_gen_ng,
                        tibble("month" = seq.Date(first, last, by="month")),
                        by = "month")

net_gen_ng %>% filter(is.na(ng_thousand_mwh))  # no missing data
```

```
## # A tibble: 0 x 2
## # i 2 variables: month <date>, ng_thousand_mwh <dbl>
```

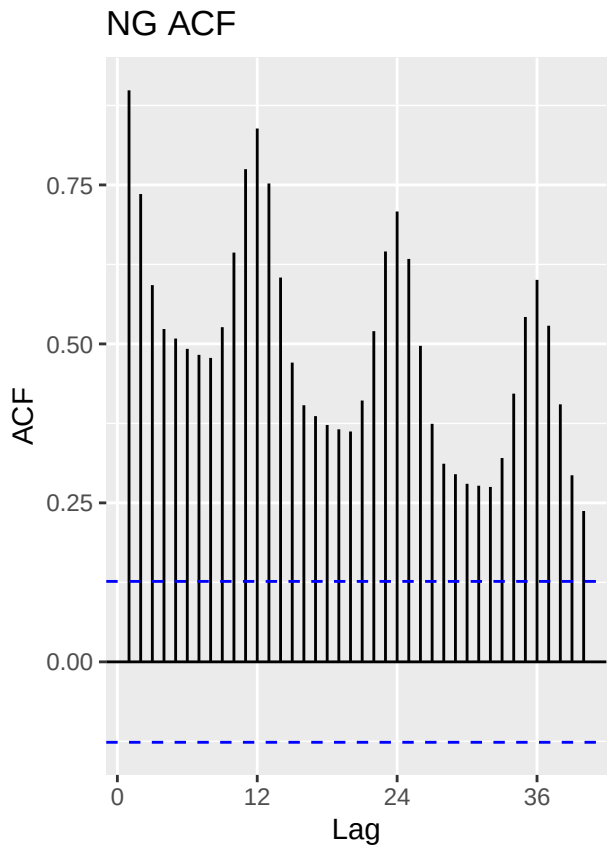
Q1

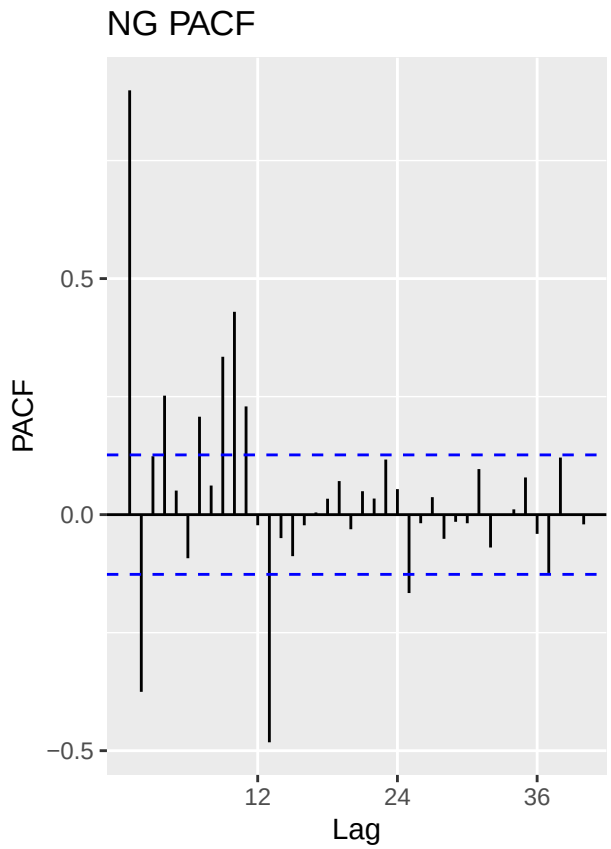
Import the csv file and create a time series object for natural gas. Make you sure you specify the **start=** and **frequency=** arguments. Plot the time series over time, ACF and PACF.

```r
ts_gen_ng <- ts(net_gen_ng$ng_thousand_mwh,
                start = c(year(first), month(first)),
                frequency = 12)

# time series plot
autoplot(ts_gen_ng) +
  theme_classic() +
  labs(x = "Month", y = "Natural Gas Generation (000s MWh)",
       title = "Monthly Natural Gas Generation (2001-2020)")
```




```r
# ACF and PACF
plot_grid(ggAcf(ts_gen_ng, lag.max = 40) + labs(title = "NG ACF"),
          ggPacf(ts_gen_ng, lag.max = 40) + labs(title = "NG PACF"))
```

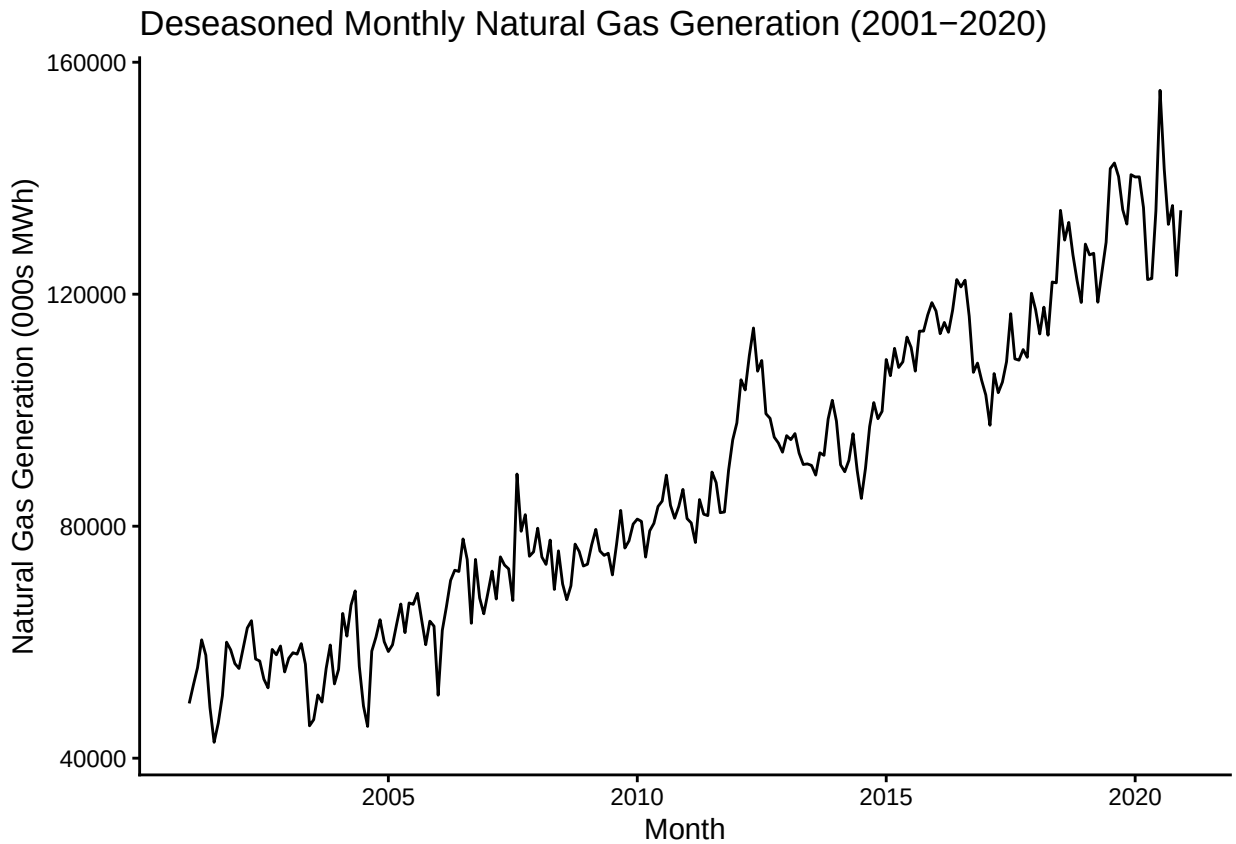




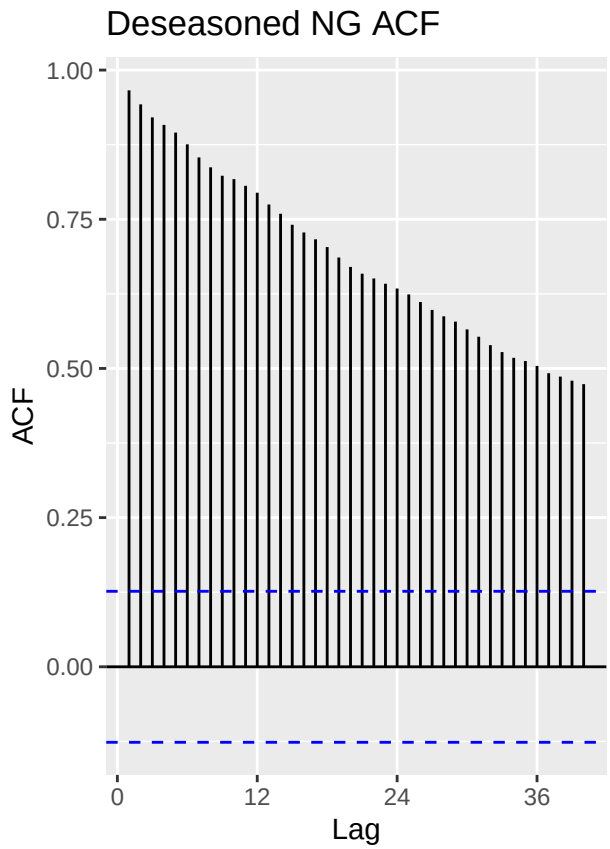
Q2

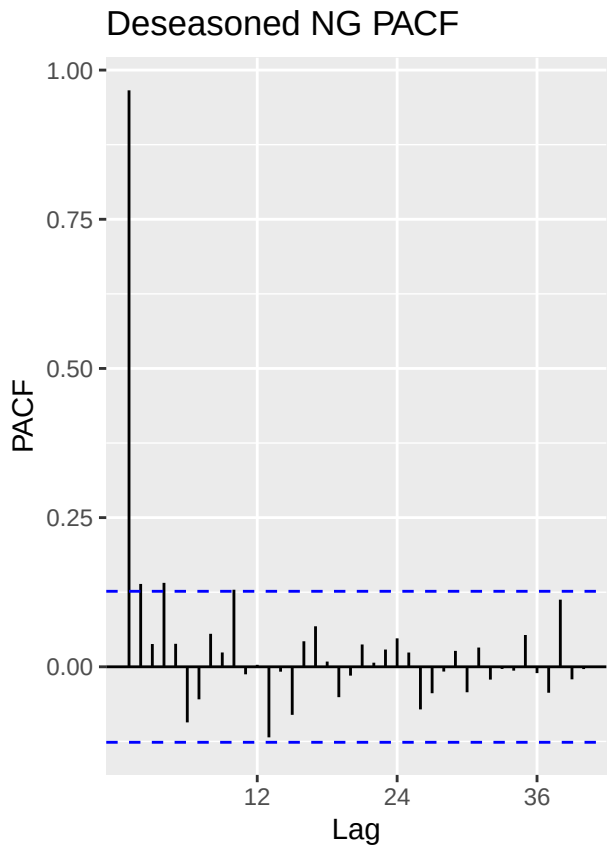
Using the *decompose*() and the *seasadj*() functions create a series without the seasonal component, i.e., a deseasonalized natural gas series. Plot the deseasonalized series over time and corresponding ACF and PACF. Compare with the plots obtained in Q1.

```
gen_ng_decomp <- decompose(ts_gen_ng)
#plot(gen_ng_decomp)

ts_gen_ng_deseason <- seasadj(gen_ng_decomp)

# plot deseasoned time series
autoplot(ts_gen_ng_deseason) +
  theme_classic() +
  labs(x = "Month", y = "Natural Gas Generation (000s MWh)",
       title = "Deseasoned Monthly Natural Gas Generation (2001-2020)")
```




```r
# plot ACF and PACF
plot_grid(ggAcf(ts_gen_ng_deseason, lag.max = 40) + labs(title = "Deseasoned NG ACF"),
          ggPacf(ts_gen_ng_deseason, lag.max = 40) + labs(title = "Deseasoned NG PACF"))
```





Comparing the time series to question 1, we no longer see clear seasonal fluctuations and movement in the series appears much more random. There is still a clear upwards trend. We no longer see the spikes every 12 lags in the ACF and PACF as we did for the original timeseries. The ACF still shows slow decay (strong autoregressive component), while the PACF shows drop off after the first lag. Both the first lag of the ACF and PACF are near 1, which suggests a unit root so we may need to difference the series. The ACF appears to still have a slight seasonality.

Modeling the seasonally adjusted (i.e., the deseasonalized series)

Q3

Run the ADF test and Mann Kendall test on the deseasonalized data from Q2. Report and explain the results.

```
# Mann-Kendall Test: Null Hypothesis = series is stationary (no trend)
mk_test_deseas <- MannKendall(ts_gen_ng_deseason)
summary(mk_test_deseas)
```

```
## Score =  24186 , Var(Score) = 1545533
## denominator =  28680
## tau = 0.843, 2-sided pvalue =< 2.22e-16
```

```r
# ADF Test: Null Hypothesis = a unit root is present
adf_test_deseas <- adf.test(ts_gen_ng_deseason)
```

```
## Warning in adf.test(ts_gen_ng_deseason): p-value smaller than printed p-value
```

```r
adf_test_deseas
```

```
##
##  Augmented Dickey-Fuller Test
##
## data:  ts_gen_ng_deseason
## Dickey-Fuller = -4.0271, Lag order = 6, p-value = 0.01
## alternative hypothesis: stationary
```

> The Mann Kendall test on the deseasoned data found that there is a significant ($p < 0.001$) trend in the data. The ADF test found that the unit root is not present ($p < 0.01$). Based on these test results, we can conclude that there is a stationary trend present in the data.

Q4

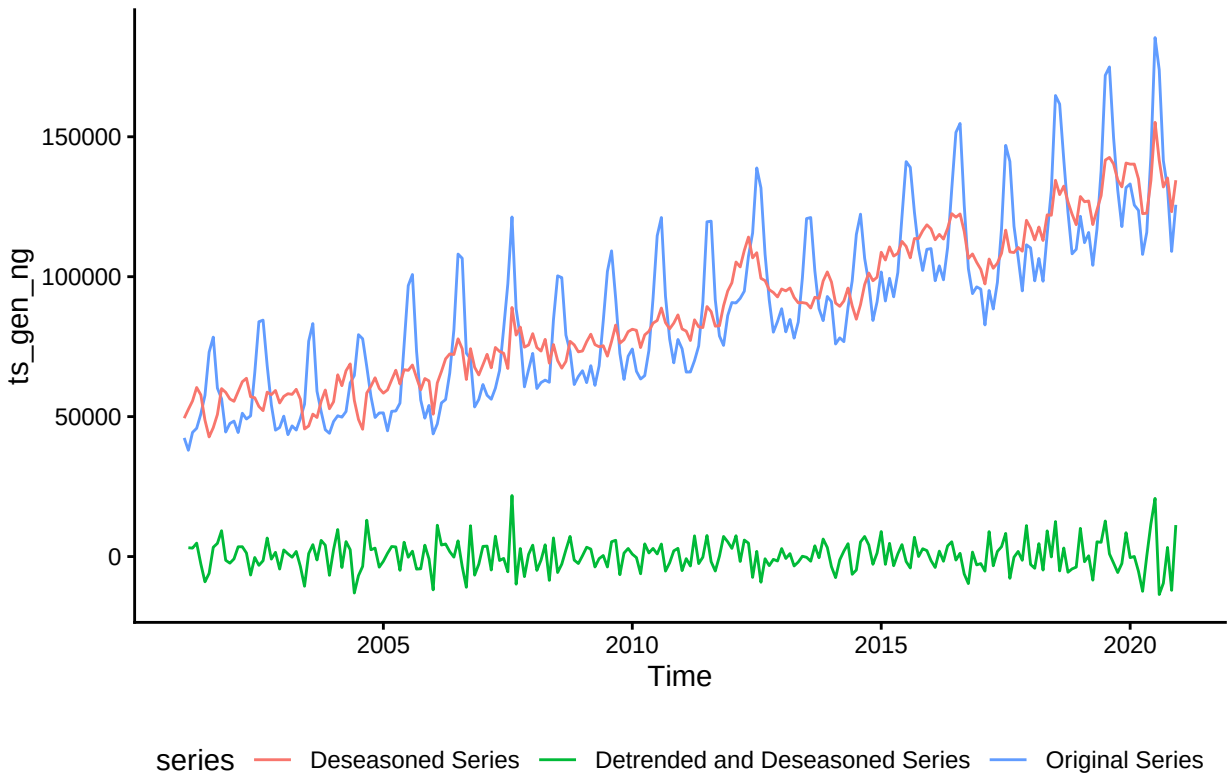
Using the plots from Q2 and test results from Q3 identify the ARIMA model parameters p, d and q. Note that in this case because you removed the seasonal component prior to identifying the model you don't need to worry about seasonal component. Clearly state your criteria and any additional function in R you might use. DO NOT use the *auto.arima*() function. You will be evaluated on ability to understand the ACF/PACF plots and interpret the test results.

> There is a trend present in the data, so we should detrend through differencing and look at the ACF/PACF plots of the stationary data to determine the model parameters.

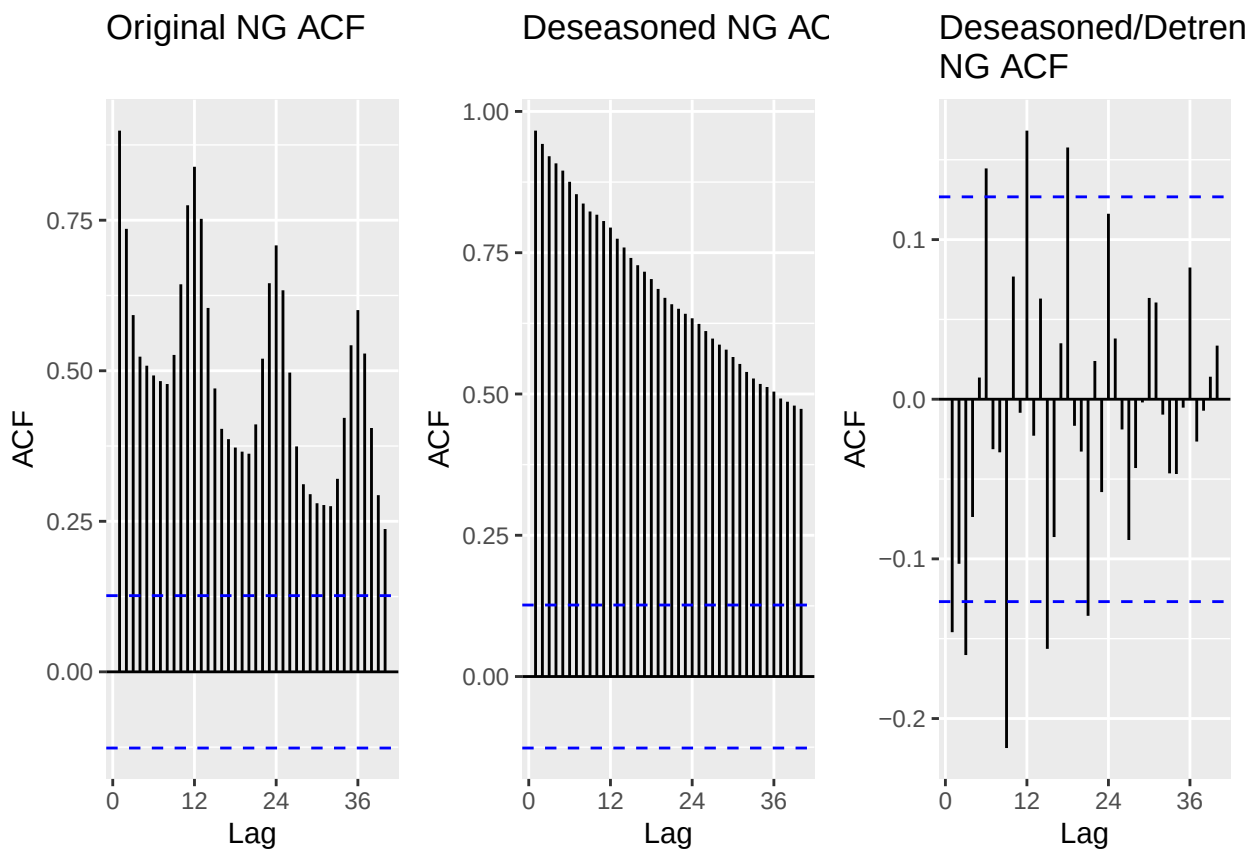
```r
ndiffs(ts_gen_ng_deseason)  # check number of differences required
```

```
## [1] 1
```

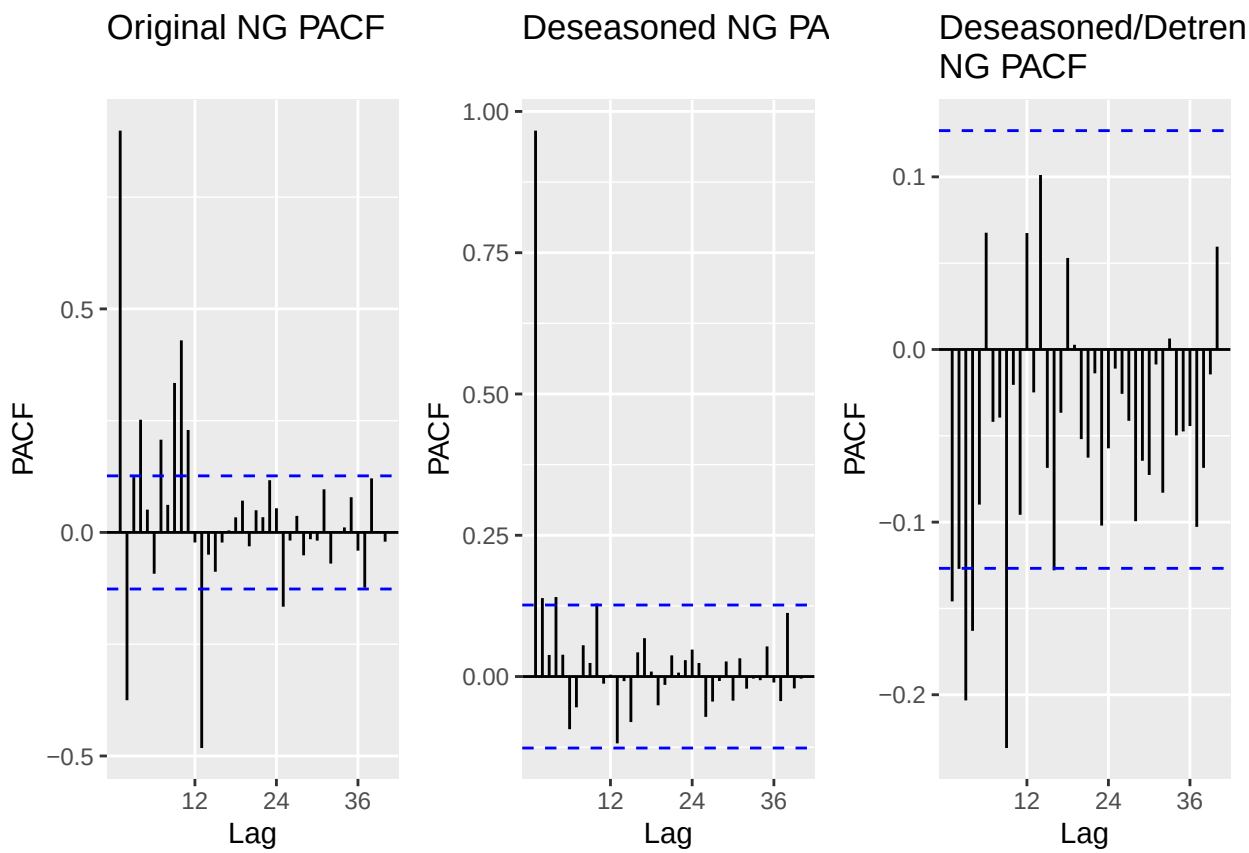
```r
#nsdiffs(ts_gen_ng_deseason)

# detrended and deseasoned time series
ts_gen_ng_detrend_deseason <- diff(ts_gen_ng_deseason, differences = 1)

autoplot(ts_gen_ng, series = "Original Series") +
  autolayer(ts_gen_ng_deseason, series = "Deseasoned Series") +
  autolayer(ts_gen_ng_detrend_deseason, series = "Detrended and Deseasoned Series") +
  theme_classic() +
  theme(legend.position = "bottom")
```




```r
# Compare ACF
plot_grid(ggAcf(ts_gen_ng, lag.max = 40) +
            labs(title = "Original NG ACF\n"),
          ggAcf(ts_gen_ng_deseason, lag.max = 40) +
            labs(title = "Deseasoned NG ACF\n"),
          ggAcf(ts_gen_ng_detrend_deseason, lag.max = 40) +
            labs(title = "Deseasoned/Detrended\nNG ACF"),
          ncol = 3)
```




```r
# Compare PACF
plot_grid(ggPacf(ts_gen_ng, lag.max = 40) +
            labs(title = "Original NG PACF\n"),
          ggPacf(ts_gen_ng_deseason, lag.max = 40) +
            labs(title = "Deseasoned NG PACF\n"),
          ggPacf(ts_gen_ng_detrend_deseason, lag.max = 40) +
            labs(title = "Deseasoned/Detrended\nNG PACF"),
          ncol = 3)
```



I used the ndiffs() function to confirm that the deseasoned series needed to be differenced once to be stationary, which is consistent with my findings from the Mann-Kendall and ADF tests. I then plotted the ACF and PACF of the deseasoned and differenced series. The ACF looks like there is an AR component present due to the exponential decay. The PACF no longer has an immediate drop off and there are a few significant lags, which may suggest the presence of an MA component as well. For this reason, I believe that this model is an ARIMA(1,1,1).

Q5

Use `Arima()` from package "forecast" to fit an ARIMA model to your series considering the order estimated in Q4. You should allow constants in the model, i.e., `include.mean = TRUE` or `include.drift=TRUE`. **Print the coefficients** in your report. Hint: use the `cat()` or `print()` function to print.

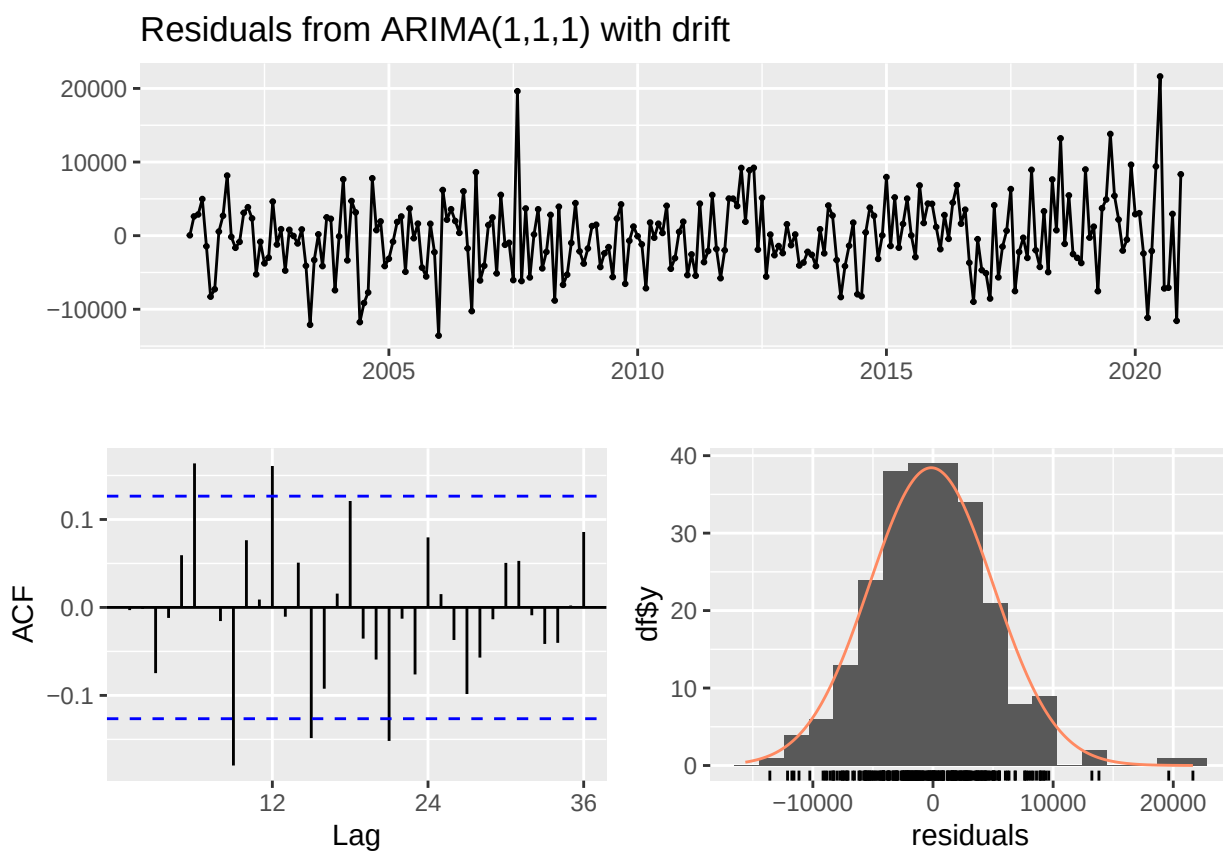
```
# use deaseasoned time series because the ARIMA will cover the differencing
model_111 <- Arima(ts_gen_ng_deseason, order = c(1,1,1),
                   include.drift = TRUE)

cat("AR1 (phi):", model_111$coef[["ar1"]],
    "\nMA1 (phi):", model_111$coef[["ma1"]],
    "\nDrift:", model_111$coef[["drift"]])
```

```
## AR1 (phi): 0.7065237
## MA1 (phi): -0.9794655
## Drift: 359.5052
```

Q6

Now plot the residuals of the ARIMA fit from Q5 along with residuals ACF on the same window. You may use the *checkresiduals*() function to get the plots. Do the residual series look like a white noise series? Why?

```
# plot_grid(
#   autoplot(model_111$residuals),
#   ggAcf(model_111$residuals, lag.max = 40)
# )
checkresiduals(model_111)    # check residuals
```




```
##
##  Ljung-Box test
##
## data:  Residuals from ARIMA(1,1,1) with drift
## Q* = 48.356, df = 22, p-value = 0.0009736
##
## Model df: 2.   Total lags used: 24
```

The residuals series looks approximately like a white noise series. There are a couple of marginally significant lags of the ACF which may suggest that the residuals are not a perfect white noise series.

Modeling the original series (with seasonality)

Q7

Repeat Q3-Q6 for the original series (the complete series that has the seasonal component). Note that when you model the seasonal series, you need to specify the seasonal part of the ARIMA model as well, i.e., P, D and Q.

Step 1: run Mann-Kendall and ADF tests to check for stationarity

```
mk_test <- MannKendall(ts_gen_ng)
summary(mk_test)     # p < 0.001, conclude that the series is not stationary (there is a significant tr
```

```
## Score =  18658 , Var(Score) = 1545533
## denominator =  28680
## tau = 0.651, 2-sided pvalue =< 2.22e-16
```

```
adf_test <- adf.test(ts_gen_ng)
```

```
## Warning in adf.test(ts_gen_ng): p-value smaller than printed p-value
```

```
adf_test             # p < 0.01, conclude that there is not a significant unit root
```

```
##
##  Augmented Dickey-Fuller Test
##
## data:  ts_gen_ng
## Dickey-Fuller = -8.9602, Lag order = 6, p-value = 0.01
## alternative hypothesis: stationary
```

```
# check for number of differences required
ndiffs(ts_gen_ng)    # d = 1
```

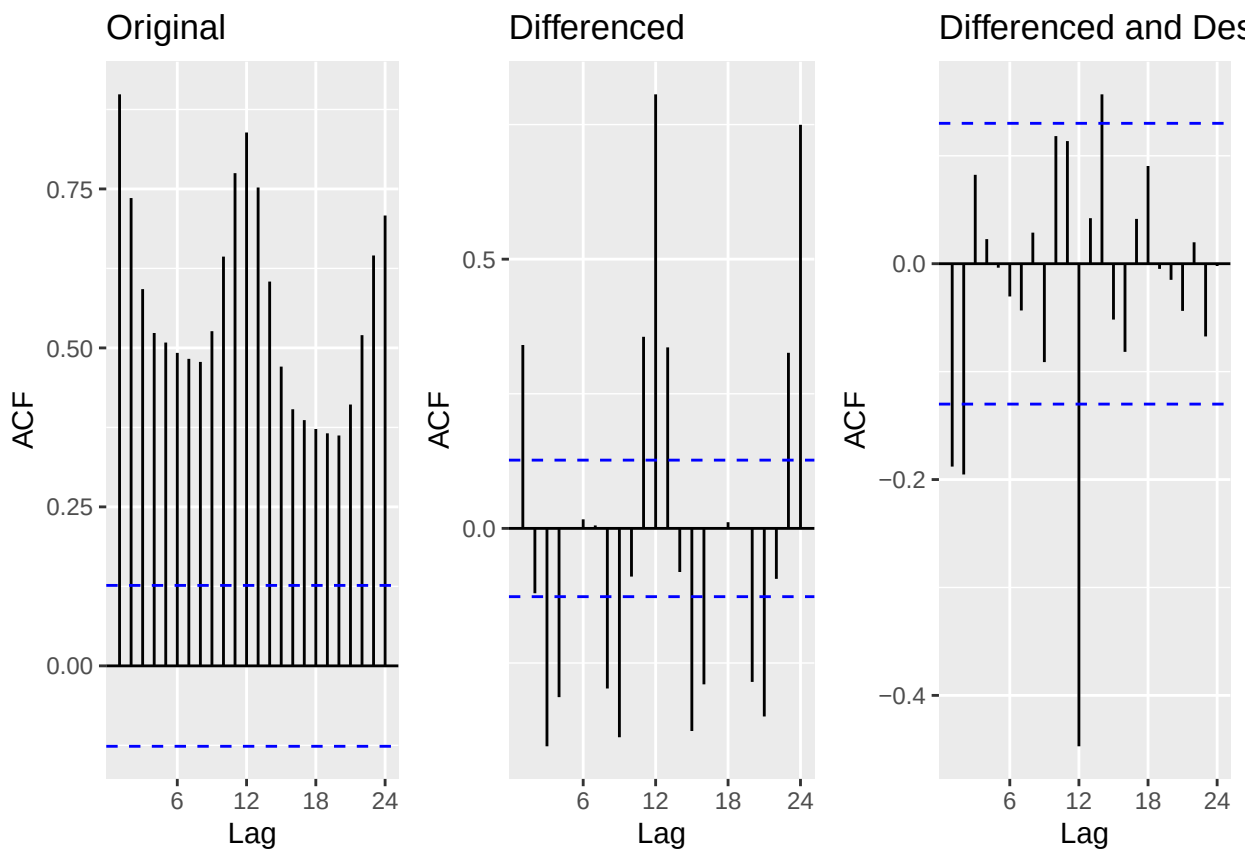
```
## [1] 1
```

```
nsdiffs(ts_gen_ng)   # D = 1
```

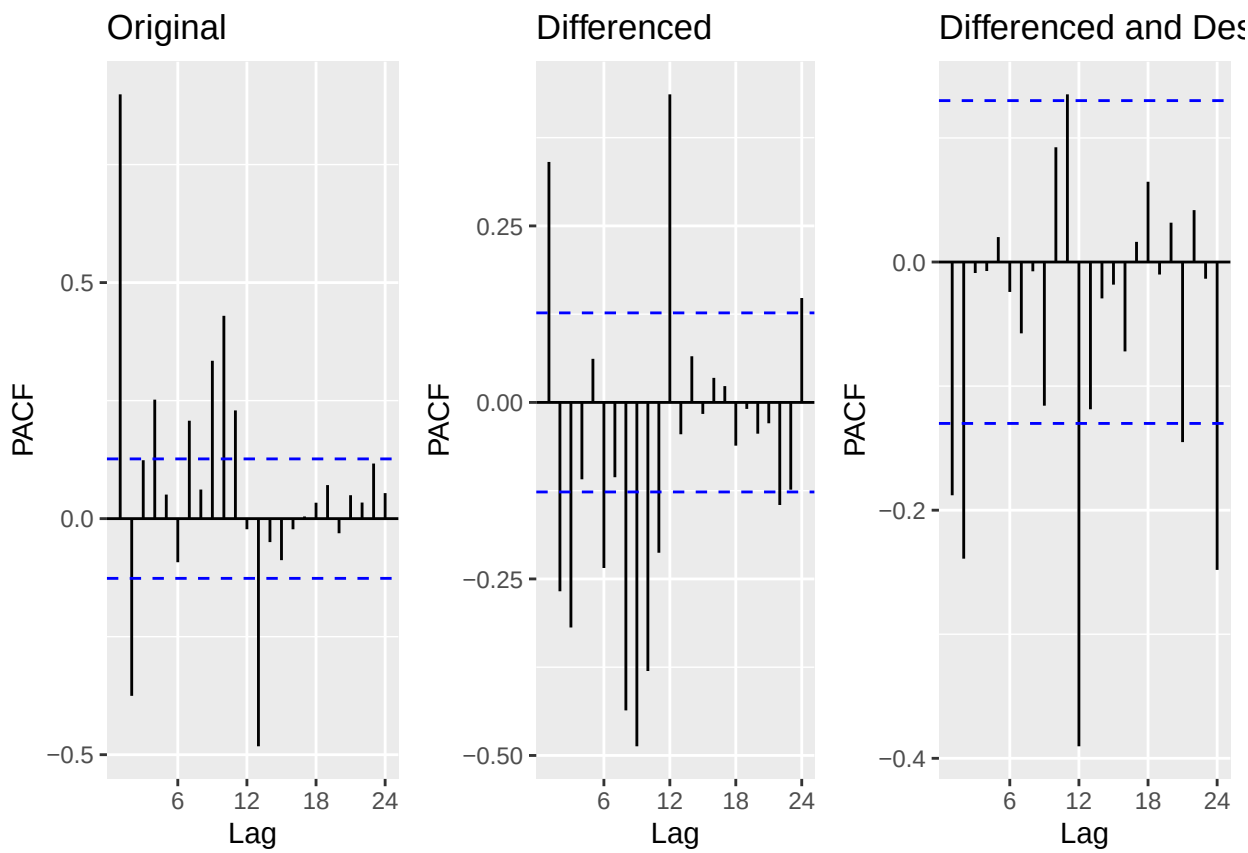
```
## [1] 1
```

Step 2: difference the series and check ACF and PACF

```
# differencing
ts_gen_ng_differenced <- diff(ts_gen_ng, differences = 1)

# seasonal differencing
ts_gen_ng_deseason_differenced <- diff(ts_gen_ng_differenced, lag=12, differences = 1)

# Compare ACF and PACF
plot_grid(ggAcf(ts_gen_ng) + labs(title = "Original"),
          ggAcf(ts_gen_ng_differenced) + labs(title = "Differenced"),
          ggAcf(ts_gen_ng_deseason_differenced) + labs(title = "Differenced and Deseasoned"),
          ncol = 3)
```




```r
plot_grid(ggPacf(ts_gen_ng) + labs(title = "Original"),
          ggPacf(ts_gen_ng_differenced) + labs(title = "Differenced"),
          ggPacf(ts_gen_ng_deseason_differenced) + labs(title = "Differenced and Deseasoned"),
          ncol = 3)
```



> Note: both the ACF and PACG

Q8

Compare the residual series for Q7 and Q6. Can you tell which ARIMA model is better representing the Natural Gas Series? Is that a fair comparison? Explain your response.

Checking your model with the auto.arima()

Please do not change your answers for Q4 and Q7 after you ran the *auto.arima*(). It is **ok** if you didn't get all orders correctly. You will not loose points for not having the same order as the *auto.arima*().

Q9

Use the *auto.arima*() command on the **deseasonalized series** to let R choose the model parameter for you. What's the order of the best ARIMA model? Does it match what you specified in Q4?

Q10

Use the *auto.arima*() command on the **original series** to let R choose the model parameters for you. Does it match what you specified in Q7?